Quiz — 2.1.5 Thinking Concurrently

Name: ___ _ Date: _ ***Mark:*** ___ / 20

OCR H446 · Component 02 · 2.1.5

Section A — Multiple choice (6 marks)

1. A weather model splits a country into regions and processes each region at the same time on a multi-core machine. This is an example of: A. Caching B. Concurrent (parallel) processing C. A precondition D. Abstraction Your answer: ☐

2. Which pair of tasks in a video-editing app are **safe to run concurrently** because they are independent? A. Rendering a clip and then exporting the very same rendered clip B. Generating audio for scene 1 and generating audio for scene 2 C. Reading a value, then writing back that value plus one D. Logging in, then loading the logged-in user's projects Your answer: ☐

3. Two concurrent threads both update a shared bank balance without coordination, and the final figure depends on which finishes first. This is a: A. Precondition B. Race condition C. Reusable component D. Decision point Your answer: ☐

4. On a computer with only a **single core**, running tasks "concurrently" actually means: A. The tasks run literally at the same instant B. The processor time-slices, rapidly switching between tasks C. The tasks are cached D. Only one task can ever be started Your answer: ☐

5. A key benefit of doing a long file download concurrently with the rest of a program is that: A. The download is guaranteed to finish instantly B. The user interface stays responsive while the download runs C. Race conditions become impossible D. The program no longer needs any inputs Your answer: ☐

6. Two processes each hold a resource the other needs and neither will release it, so both wait forever. This is: A. Caching B. Deadlock C. Decomposition D. A precondition Your answer: ☐

Section B — Short answer (9 marks)

7. An online store generates a sales report by (i) totalling each region's sales and (ii) combining the regional totals into a grand total. Explain which part could run concurrently and which must run afterwards, referring to data dependency. **[3]**

8. Discuss **one** benefit and **one** trade-off of using concurrency to load the images on a busy news website. **[2]**

9. Explain why running a task concurrently is **not always faster**, giving one reason. **[2]**

10. State what a *race condition* is and give one example of where it could occur in a shared online ticket-booking system. **[2]**

Section C — Applied question (5 marks)

11. A photo-sharing app must, when a user uploads a photo: (a) generate three different thumbnail sizes, (b) scan the image for prohibited content, and (c) save the original to storage. Using *thinking concurrently*, identify which of these tasks could run at the same time and which must be sequenced, then discuss **one** benefit and **one** trade-off of running them concurrently. **[5]**

Answer key

Section A (6 marks — 1 each) 1. **B** — independent regions processed simultaneously on multiple cores is parallel/concurrent processing. 2. **B** — scene 1 and scene 2 audio are independent (no shared data), so they can run together; the others have dependencies. 3. **B** — an outcome that depends on unpredictable timing of threads sharing data is a race condition. 4. **B** — a single core cannot run tasks truly simultaneously; it time-slices between them. 5. **B** — offloading the download keeps the UI responsive while it runs. 6. **B** — mutual waiting on each other's held resources is deadlock.

Section B (9 marks)

7. **[3]** The regional totals can be computed concurrently because each region's figure is independent of the others (no shared data) (1). Combining them into a grand total must run **after** all regional totals exist (1), because it depends on (uses the output of) those totals — a data dependency forces it to be sequential (1).

8. **[2]** Benefit (1): images load in parallel so the page appears/completes faster and the page stays responsive while they load. Trade-off (1): harder to program/debug, more overhead, and on a single core it is time-sliced rather than truly parallel so the gain may be limited; results may need synchronising. Must be applied to the image-loading scenario.

9. **[2]** Any one reason (2, or 1+1): on a single core there is no true parallelism — just time-slicing — so no real speed-up (1); there is overhead in creating/managing threads and synchronising/recombining results (1); some problems are inherently sequential because later steps depend on earlier outputs.

10. **[2]** A race condition is where the result depends on the unpredictable order/timing of concurrent tasks that share data (1). Example (1): two customers' processes both read "1 seat left" at the same moment and both book it, so the same seat is sold twice.

Section C (5 marks) 11. **[5]** Up to 5 from: Tasks (a) generate-thumbnails, (b) scan-content and (c) save-original are largely independent of each other (each works from the uploaded image), so they could run concurrently (1). A sensible dependency may be noted — e.g. the app may need a valid/safe image

before it is *published*, so the content scan result may have to be checked before the photo is shown, even if the work runs in parallel (1). Benefit (1): doing them at once makes the upload feel faster / keeps the app responsive and uses multiple cores. Trade-off (1): concurrency is harder to design and debug, risks race conditions if they touch shared storage, and results may need synchronising; on limited hardware it may not be truly faster (1). Must apply to the upload scenario and weigh both benefit and trade-off.

Total: 6 + 9 + 5 = 20